07｜上下文的艺术（下）：用 constitution.md 为AI注入项目“宪法”

Tony Bai · AI 原生开发工作流实战



你好，我是 Tony Bai。

在上一讲，我们一起为我们的 Go 项目精心编写了一份 `CLAUDE.md` 参考模板文件。这份文件就像一本详尽的“**团队工作手册**”，它告诉了 AI Agent 关于项目的技术栈、构建命令、代码风格、Git 工作流等所有操作性的细节。

学完之后，你可能会觉得，我们的 AI 伙伴已经足够“懂行”了。它知道该用什么命令去测试，知道该用什么格式去提交代码。但是，一些新的、更深层次的问题浮现了：

> 当面对一个需求，AI Agent 提出了两种技术方案——一个简单直接但可能留下技术债，另一个结构优雅但实现复杂，**它应该如何抉择？**
>
> 当为了快速实现功能，AI Agent 想要引入一个新的、我们团队从未用过的第三方库时，**它应该被允许吗？**

> 当 AI Agent 生成的代码虽然通过了所有测试，但其设计模式却与我们项目长期坚持的“保持简单、避免过度抽象”的哲学相悖时，**我们该如何约束它?**

这些问题，已经超出了“工作手册”能够回答的范畴。它们不关乎“如何做”，而关乎“应该怎样做”和“绝不能怎样做”。它们触及了一个项目的核心价值观和架构哲学。

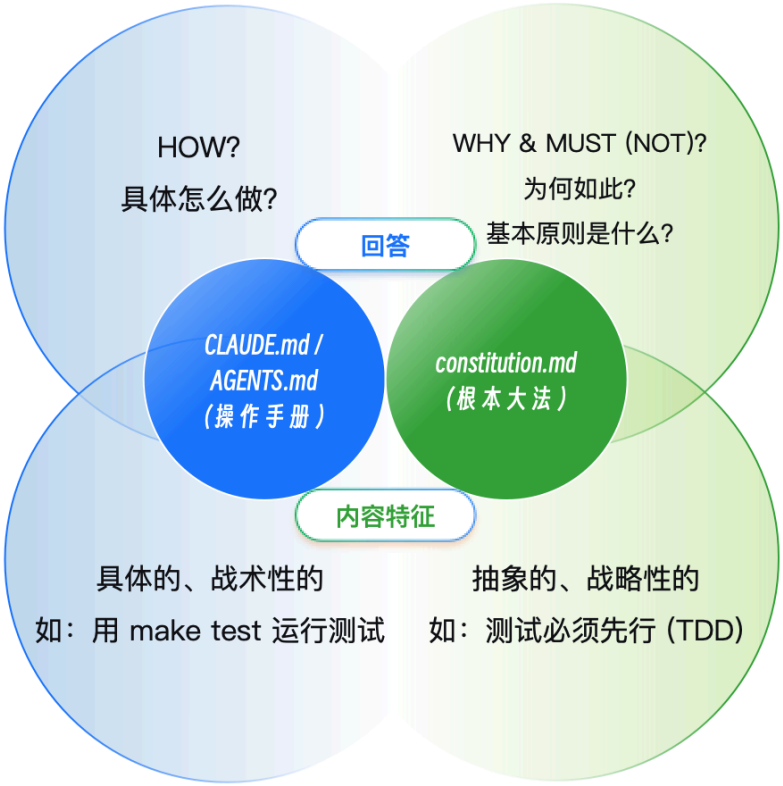
今天这一讲，我们将完成“上下文的艺术”的最后一块拼图。我们将从“术”的层面，上升到“道”的层面，学习如何为我们的 AI Agent 和项目，注入一个不可动摇的“灵魂”——那就是源自 `spec-kit` 前沿思想的 `constitution.md`，即项目“宪法”。

从“工作手册”到“根本大法”：两种上下文的定位差异

在深入 `constitution.md` 的具体内容之前，我们必须首先清晰地辨析它与我们在上一讲学习的 `CLAUDE.md`/`AGENTS.md` 之间的根本区别。

如果说 `CLAUDE.md` 是交给 AI Agent 的“操作指南”，那么 `constitution.md` 就是我们与 AI Agent 共同签署的“原则契约”。

两种“长期记忆”的定位对比



极客时间

它们的差异体现在三个核心维度：一是**抽象层级不同**，`CLAUDE.md` 是低层级的、具体的。它直接告诉 AI Agent“运行这个命令”“遵循这个格式”。`constitution.md` 是高层级的、抽象的。它不关心具体的命令，而是定义了“测试必须先于代码”“保持简单，避免不必要的抽象”等根本性原则。

二是**强制力不同**，`CLAUDE.md` 是指导性的。AI Agent 会参考它来完成任务，但在某些情况下可能会有变通。`constitution.md` 具有绝对的否决权。任何与“宪法”中“NON-NEGOTIABLE”（不可协商）条款相冲突的计划或实现，都应被视为“违宪”而被驳回。AI Agent 在进行技术规划时，必须首先对照“宪法”进行自检。

三是**可变性不同**，`CLAUDE.md` 是相对易变的。随着我们引入新的工具、升级依赖，这份“操作手册”会经常更新。`constitution.md` 是高度稳定的。它代表了项目的核心技术哲学，除非项目发生重大的战略转向，否则不应轻易修改。

理解了这种“**宪法 > 指南**”的层级关系，你就能明白，一份好的 `constitution.md`，是 AI Agent 在面对复杂决策时的北极星，是确保 AI Agent 所有创造性工作始终与项目长期利益保持一致的压舱石。

解读“开发九章”：深入 `spec-kit` 的架构哲学

`constitution.md` 这个概念，最早由 GitHub 的 `spec-kit` 项目提出并系统化。它并非一份随意的文档，而是一套经过深思熟虑的、旨在约束 AI Agent 行为、确保工程质量的**架构哲学**。`spec-kit` 的规范驱动开发说明中，这份“宪法”被分为了九条核心“法案”来举例说明。虽然我们不必全盘照搬，但深入理解这“开发九章”示例背后的思想，对于我们编写自己的“宪法”至关重要。

原则	内容	解读
库先行原则 Library-First Principle	每一个功能都必须首先作为一个独立的库来实现，绝不能直接在应用代码中实现	这是在强制AI Agent从一开始就进行**模块化和高内聚、低耦合的设计**。它避免了AI Agent为了快速实现功能而将所有逻辑都堆砌在一起，从而产生难以维护的“大泥球”代码
CLI接口强制 CLI Interface Mandate	每一个库都必须通过命令行接口（CLI）暴露其核心功能	这一条极其精妙，它强制了**可测试性**和**可观测性**。当一个功能可以通过CLI调用时，它就天然地易于被自动化测试，也易于被其他脚本或工作流编排。它杜绝了AI Agent创造出那些隐藏在复杂GUI交互背后的、难以验证的“黑盒逻辑”
测试先行铁律 Test-First Imperative	**不可协商**：所有实现都必须遵循严格的测试驱动开发（TDD）。在单元测试被编写、被用户批准、并被确认为**失败**（Red）之前，不得编写任何实现代码	这是spec-kit哲学中最激进也最核心的一条。它彻底颠覆了“AI生成代码，人来补测试”的传统协作模式。它要求AI Agent首先扮演一个**测试工程师**的角色，精确地定义一个功能“应该做什么”，然后再扮演**开发工程师**的角色去实现它。这极大地保证了代码的正确性和鲁棒性
第四章至第六章 集成、可观测性、版本控制	这些条款进一步细化了工程实践，例如强调集成测试的重要性、要求结构化日志、定义清晰的版本号规范等	-
第七、八章 简单性与反过度抽象原则 Simplicity and Anti-Abstraction	“初始实现的项目结构不能超过3个”、“直接使用框架特性，而不是过度包装”、“保持单一的模型表示”	这两条是专门为了对抗大模型一个常见的“坏习惯”——**过度工程（Over-engineering）**。AI有时会为了展现其“智能”，而设计出过于复杂、充满不必要抽象的设计模式。这两条“宪法”就像两道“门禁”，强制AI在提出任何复杂方案时，都必须给出充分的理由
集成优先测试 Integration-First Testing	测试必须使用真实环境，优先使用真实的数据库而非Mocks	这条原则强调了测试的有效性，确保AI生成的代码能够在真实世界中正常工作，而不仅仅是在理想化的、被Mock的单元测试环境中

第一章是库先行原则：每一个功能都必须首先作为一个独立的库来实现，绝不能直接在应用代码中实现。”这是在强制 AI Agent 从一开始就进行模块化和高内聚、低耦合的设计。它避免了 AI Agent 为了快速实现功能而将所有逻辑都堆砌在一起，从而产生难以维护的“大泥球”代码。

第二章是 CLI 接口强制：“每一个库都必须通过命令行接口（CLI）暴露其核心功能。”这一条极其精妙，它强制了可测试性和可观测性。当一个功能可以通过 CLI 调用时，它就天然地

易于被自动化测试，也易于被其他脚本或工作流编排。它杜绝了 AI Agent 创造出那些隐藏在复杂 GUI 交互背后的、难以验证的“黑盒逻辑”。

第三章测试先行铁律：不可协商，所有实现都必须遵循严格的测试驱动开发。在单元测试被编写、被用户批准、并被确认为失败之前，不得编写任何实现代码。”这是 `spec-kit` 哲学中最激进也最核心的一条。它彻底颠覆了“AI 生成代码，人来补测试”的传统协作模式。它要求 AI Agent 首先扮演一个测试工程师的角色，精确地定义一个功能“应该做什么”，然后再扮演开发工程师的角色去实现它。这极大地保证了代码的正确性和鲁棒性。

第四章至第六章（集成、可观测性、版本控制）：这些条款进一步细化了工程实践，例如强调集成测试的重要性、要求结构化日志、定义清晰的版本号规范等。

第七、八章是简单性与反过度抽象原则：“初始实现的项目结构不能超过 3 个”“直接使用框架特性，而不是过度包装”“保持单一的模型表示”。这两条是专门为了对抗大模型一个常见的“坏习惯”——过度工程。AI 有时会为了展现其“智能”，而设计出过于复杂、充满不必要抽象的设计模式。这两条“宪法”就像两道“门禁”，强制 AI 在提出任何复杂方案时，都必须给出充分的理由。

第九章集成优先测试：“测试必须使用真实环境，优先使用真实的数据库而非 Mocks。”这条原则强调了测试的有效性，确保 AI 生成的代码能够在真实世界中正常工作，而不仅仅是在理想化的、被 Mock 的单元测试环境中。

这“开发九章”，共同构成了一套严谨的、旨在引导 AI 生成高质量、可维护、可测试代码的顶层设计哲学。

实战：为我们的 Go 项目量身定制一份 `constitution.md`

现在，让我们吸收 `spec--kit` 的思想精髓，为我们的 Go 项目 `issue2md` 量身定制一份更简洁、更符合 Go 语言社区风格的 `constitution.md`。

这份文件，我们将放置在项目的根目录下，或者 `./.claude/` 目录中，并通过上一讲学到的 `@` 导入语法，在我们的 `CLAUDE.md` 中引入它，以确保 AI Agent 始终能“看到”它。

`constitution.md` for `issue2md`

复制代码

```
# issue2md 项目开发宪法
# Version: 1.0, Ratified: 2025-10-20

本文件定义了本项目不可动摇的核心开发原则。所有AI Agent在进行技术规划和代码实现时，必须无条件遵循

---

## 第一条：简单性原则 (Simplicity First)

**核心：** 遵循Go语言的“少即是多”哲学。绝不进行不必要的抽象，绝不引入非必需的依赖。

- **1.1 (YAGNI):** 你不需要它（You Ain't Gonna Need It）。只实现`spec.md`中明确要求的功
- **1.2 (标准库优先):** 除非有极其充分的理由，否则必须优先使用Go标准库。例如，Web服务使用`net
- **1.3 (反过度工程):** 避免复杂的设计模式。简单的函数和数据结构优于复杂的接口和继承体系。

---

## 第二条：测试先行铁律 (Test-First Imperative) - 不可协商

**核心：** 所有新功能或Bug修复，都必须从编写一个（或多个）失败的测试开始。

- **2.1 (TDD循环):** 严格遵循“Red-Green-Refactor”（编写失败测试-让测试通过-重构）的循环。
- **2.2 (表格驱动):** 单元测试必须优先采用表格驱动测试（Table-Driven Tests）的风格，以覆盖
- **2.3 (拒绝Mocks):** 优先编写集成测试，使用真实的依赖或fake object（如内存中的GitHub AP

---

## 第三条：明确性原则 (Clarity and Explicitness)

**核心：** 代码的首要目的是让人类易于理解，其次才是让机器执行。

- **3.1 (错误处理):** **不可协商**：所有错误都必须被显式处理。绝不允许使用`_`丢弃错误。错误传
- **3.2 (无全局变量):** 绝不允许使用全局变量来传递状态。所有依赖必须通过函数参数或结构体成员显
- **3.3 (注释的意义):** 注释应该解释“为什么”，而不是“是什么”。所有公共API都必须有清晰的GoDoc

---

## 第四条：单一职责原则 (Single Responsibility)

**核心：** 每个包、每个文件、每个函数都应该只做好一件事。

- **4.1 (包的内聚):** `internal`目录下的各个包应保持高度内聚和低耦合。例如，`github`包只负
- **4.2 (接口隔离):** 定义小的、目标明确的接口，而不是大而全的“上帝接口”。

```

```
---
## 治理 (Governance)

本宪法具有最高优先级，其效力高于任何`CLAUDE.md`或单次会话中的指令。任何计划（`plan.md`）在生成
```

这份为 Go 项目量身定制的“宪法”，既吸收了 `spec-kit` 的精华（如 TDD、简单性），又融入了 Go 社区的最佳实践（如标准库优先、明确的错误处理）。它将成为我们后续所有实战中，约束和引导 AI 行为的最高准则。

AI 如何理解并遵循“宪法”？

你可能会问，AI Agent 真的能理解并像人类一样遵循这些抽象的原则吗？答案是肯定的，但这需要一个 **“机制 + 引导”** 的双重保障。

** 机制保障（通过 `plan.md` 模板）： ** 最可靠的方式，是将“合宪性审查”变成一个强制性的流程节点。在 `spec-kit` 的实践中，`plan.md` 的模板里包含了一个名为“Constitution Check”的章节，里面有多个“Gate（门禁）”。

例如，一个 `plan.md` 的模板可能是这样的：

复制代码

```
## Constitution Check (合宪性审查)

*GATE: 必须在进行技术方案设计前通过。*

- **[ ] 简单性门禁 (第一条):** 是否优先使用了标准库？是否避免了不必要的抽象？
- **[ ] 测试先行门禁 (第二条):** 计划中是否包含了“先写测试”的步骤？
- **[ ] 明确性门禁 (第三条):** 是否明确了依赖注入的方式，而非全局变量？

*如果任何门禁未通过，必须在下方“复杂性追踪”部分给出充分理由。*
```

当 AI Agent 被要求生成 `/plan` 时，它看到这个模板，就会被强制地去对照 `constitution.md` 的条款，逐一进行检查和打勾。这就将抽象的原则，转化为了一个具体、可执行的清单。

引导保障（通过 `CLAUDE.md` 和提示）： 我们还可以在 `CLAUDE.md` 中，明确地提醒 AI。就像我们上面编写的：

> 当被要求添加新功能时：你的第一步应该是先用 `@` 指令阅读 `internal/` 下的相关包，并对照 `constitution.md` 的原则，然后再提出你的计划。

通过这种“机制 + 引导”的方式，我们就为“宪法”的执行，上了一道双保险。

本讲小结

今天，我们完成了“上下文的艺术”的升维一跃。我们不再满足于告诉 AI 具体“怎么做”，而是学会了如何定义它“应该成为什么”。

首先，我们深度辨析了 `CLAUDE.md`（操作手册）与 `constitution.md`（根本大法）在定位、抽象层级、强制力和可变性上的根本区别，明确了“道”与“术”的分工。接着，我们深入解读了 `spec-kit` 中极具前瞻性的“**开发九章**”，理解了其背后深刻的架构哲学和工程思想。

然后，也是最重要的一步，我们亲手为我们的 Go 项目量身定制了一份简洁而强大的 `constitution.md`，将 Go 社区的最佳实践内化为 AI Agent 必须遵循的铁律。最后，我们探讨了如何通过“**机制 + 引导**”的双重保障，来确保 AI Agent 能够真正地理解并遵循我们制定的“宪法”。

掌握了 `constitution.md`，你就掌握了在 AI 原生开发中，进行顶层设计和质量治理的终极武器。你不再仅仅是 AI 的使用者，而是成为了 AI 行为的立法者和监督者。

至此，我们已经为 AI 伙伴构建起了一套完整、分层的“长期记忆系统”。从下一讲开始，我们将进入更激动人心的领域——**AI 能力扩展**。我们将从最简单也最实用的 Slash Commands 开始，学习如何为 AI 添加全新的技能。

思考题

请你审视一下你当前所在团队或你参与的开源项目，是否存在一些不成文的、但大家都在默默遵守的“开发哲学”或“架构原则”？例如，“我们绝不使用 ORM”“所有的数据库变更都必须有

回滚脚本”“前端组件必须是无状态的”等等。

请你尝试将其中最重要的一到两条原则，用我们今天学习的 `constitution.md` 的风格，写成“法案”的形式。欢迎在评论区分享你的“立法”尝试。这个练习将帮助你思考如何将团队的隐性知识，转化为可被 AI 理解和执行的显性规则。如果你觉得有所收获也欢迎你分享给需要的朋友，我们下节课再见！

© 版权归极客邦科技所有，未经许可不得传播售卖。 页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (29)

最新　精选



瓶子
2025-12-15 来自北京

规范是不是不宜过大？现在团队内都在“无节制”地往 项目级规范中新增内容……

> 作者回复: 团队不断丰富和优化项目规范是正确的。但就我的实践来看，有几点对抗“规范膨胀”的小tip吧：
>
> – 不要把所有规范塞进一个文件，至少保证单一的文件没那么膨胀。可以按领域划分到不同规范文件中，比如 go-style.md, db-rules.md等，然后在CLAUDE.md或AGENT.md中import。
> – 尽量工具优先。凡是能用 像golangci-lint、ESLint 等自动化工具解决的规范，可考虑不必写在 Prompt 或 Markdown 里。让工具去管，而不浪费 AI 的 Token 去读规范。
> – 定期“优化”： 团队应定期审查，把过时的、过于琐碎的规范剔除。

共 3 条评论 >　　16



Biluota
2025-12-04 来自陕西

我了解了一下 spec-kit ，该项目可以直接本地安装 spec-kit ，在本地项目中通过 'specify init .'命令，并选择要使用的 AI assistant (比如 cc)，就可以进行相关规范标准生成。然后在项目根目录下运行 claude，输入'/speckit.constitution'就可以自动生成 consititution.md 文件，'/speckit.specify 自然语言的功能描述'就可以进行在宪法条件下的功能开发了。请问这

种方式和老师您说的宪法使用方式有什么异同吗，通过这种方式生成的 constitution 文件是否可取，这种方式实现的功能开发有什么需要注意的吗

作者回复: 相同点是核心思想是一致的，在专栏前面我就讲过，专栏基于的就是 spec-kit 背后的 SDD 方法论。

不同的是spec-kit是“工具化”的实现。它把流程封装成了 CLI 命令（比如specify init），帮你自动生成模板、自动配置 Slash Commands。优点是快，缺点是黑盒，不够灵活。

而我们的专栏，尤其是后面实战篇。是“原生/手动”的实现。讲解让你如何从零手写 CLAUDE.md 和 constitution.md。优点是你能彻底理解原理，并能根据自己的项目，缺点是前期配置稍繁琐。

如果你不想折腾配置，学完本专栏后，也可以直接用 spec-kit 生成的基础文件也是很好的起点。

但不要迷信工具生成的默认值。 spec-kit 生成的 constitution.md 可能是通用的。你必须去review，在必要时还需修改，把它改成符合你团队（比如 Go 项目）实际情况的“宪法规则”（如“优先用标准库”），否则它起到的作用可能就不明显。

共 2 条评论 >

14



Geek4329

2025-12-03 来自四川

请问老师，假设一个需求会涉及变更3个服务，这3个服务分别有各自的git仓库，那么在用AI去生成技术方案、编码时，怎么做到让AI全局考虑到这3个服务，有比较好的落地实践吗。
当前我的想法是，如果一个需求涉及3个领域的业务服务还好拆分，然后分给不同的服务，各自用AI去处理。
但是如果要改动的服务都属于一个业务领域，比如有2个服务，一个服务的产品层服务（鉴权、调用核心接口），一个核心层服务，那么不好通过拆分PRD去给到不同服务去处理

作者回复: 你的思路没错。如果要解决属于同一个业务领域的跨服务的“全局思考”问题，有时候还真需要让ai建立全局的“合体认知”。

如果你公司用的是monorepo，那是解决“合体认知”最天然的结构。

如果不是monorepo，而是分别在不同的repo，那可以试试在本地创建一个包含这三个服务代码的统一工作区（可以通过 git submodule 或简单的文件夹聚合）。在这个统一工作区根目录下，编写 spec.md 和 plan.md。有了全局的 Plan 后，再拆解成三个独立的 tasks.md，分别下发给针对每个服务的 Agent 会话（或者在一个会话中依次切换目录执行）。

你可以试验一下，也欢迎在留言区分享你试验后的心得。

共 3 条评论 >

12



yunplane

2025-12-03 来自湖南

最后讲到的机制 + 引导的双重保障，不太明白，麻烦老师再详细点

作者回复: 简单来说，就是“硬卡口”和“软提醒”的结合。

硬卡口：这是写在 plan.md 模板里的“填空题”。当 AI 生成计划时，它被迫要在这个方框里打钩。为了打这个钩，它就不得不真的去检查一遍宪法。

软提醒: 就是写在 CLAUDE.md 里的“教导”。比如你告诉 AI：“你在做设计之前，要先去看看 constitution.md 哦”。 让 AI 知道有这个规则存在。因为 AI 可能会忘。

因此，所谓 双重保障 = 逼它画勾确认（机制） + 告诉它要检查（引导）。

共 3 条评论 >

9



6点无痛早起学习的和...

2026-01-27 来自北京

看完了，回到这个章节来提问
其实发现Claude code的经常性忘记配置好的Claude.md和 consitution.md文件里的内容

作者回复: 随着对话变长，上下文窗口被挤占，早期的 Prompt（包括自动加载的文件）会被压缩或丢弃。

可以通过/compact定期压缩无关对话，给核心规则腾地方。

在关键指令里显式引用：“请严格对照 constitution.md 执行...”。这能重新激活 AI 对规则的注意力。

此外，不要在一个session里做太多事儿！开一个新session应付新的工作。

4



s(t)
2026-01-08 来自上海

这章和上章的思考题有啥区别吗？看着都是把一些规则放到 md 里面。是 CLAUDE.md 里放一些具体的、写代码的规则，比如我从map中取值必须给默认值，在宪章里面是一些比较抽象的架构层面的规则，比如不要用状态管理这种吗

作者回复: 你的理解非常正确！

违背 CLAUDE.md 是“代码风格不好”，违背 constitution.md 是“架构设计错误”。后者是红线，前者是规范。

共 2 条评论 >

4



jarvis
2025-12-14 来自上海

实话说，我就会一点点的python，就是纯业余爱好，发现用speckit流程做出相关文档，再用一些差一点的模型，也能生成差不多的代码了，至少证明现在的模型能力，文档规范驱动还是有效的，远远强于我之前和llm聊天模式的codeing。

作者回复: 👍

4

Geek_95454a
2025-12-03 来自浙江

老师，还是不太确定 constitution.md 是需要在 claude,md 中引入吗？最终的例子中没有具体表现的；比方说我现在正常进行开发，claude.md 上应该是没有像 plan.md 那种Constitution Check (合宪性审查)；我是需要在每个项目的 claude 中追加这个一段审查吗？还是可以通过某些方法，每一次增加能力，都必然会走这个审查呢?

作者回复: 是否需要引入？如果你使用constitution.md，那就 必须在claude.md中引入。 AI 不会自动知道 constitution.md 的存在。

如何确保每次都审查？仅仅“存在”是不够的。 就像法律书摆在书架上，人不一定会看。一种方法是需要在提示词或模板中指定 类似 Constitution Check。这个在后面的实战篇讲解中有相应的示例。

4



熊出没

2026-03-04 来自湖北

明白了 constitution.md还是需要CLAUDE.md来加载 也就是说Claude code 只认CLAUDE.md 通过引用加载 所以它叫不叫constitution.md关系不大

作者回复: 理解正确。

2

HaHahahah

2026-02-12 来自北京

老师，请问下之前都在写project_rules.md，请问这个和宪章、CLAUDE.md之间的关系是什么。我要定义项目规范，如分层结构、异常处理，这些是写入到宪章还是rules。

作者回复: project_rules.md ≈ CLAUDE.md， 它们都是“操作手册”。很多 Cursor 用户习惯叫 rules，Claude 用户习惯叫 CLAUDE.md。它们放的是具体的、战
术级的规范。

分层结构、异常处理原则（如“必须分层”、“禁止吞异常”）等放入 constitution.md（宪法）。这是架构原则，不可动摇。

具体写法（如“Controller 层文件命名必须以_ctl结尾”、“用 errors.Wrap”）： 放入 CLAUDE.md。这是执行细节，方便 AI 照抄。

2